

Avance 6

Reporte de Depuración y Evaluación de Componentes de Software

David Armando Ramírez Navarrete

04 de febrero de 2026

Resumen

El presente informe técnico documenta los hallazgos obtenidos durante la revisión inicial de la documentación de Looker Studio, como parte del análisis de información realizado en las primeras semanas de prácticas. El trabajo refleja habilidades de análisis, revisión estructurada de documentación técnica y atención al detalle.

Introducción

Las prácticas profesionales representan una etapa formativa fundamental para la aplicación de los conocimientos adquiridos durante la formación académica en un entorno técnico-profesional. A través de estas actividades, es posible analizar procesos reales, identificar áreas de oportunidad y proponer soluciones orientadas a la mejora de la eficiencia operativa, manteniendo siempre un enfoque responsable en el manejo de la información.

En este contexto, la automatización de tareas operativas constituye una estrategia clave para optimizar procesos que se caracterizan por ser repetitivos, manuales y propensos a errores humanos. La implementación de scripts de automatización permite reducir tiempos de ejecución, estandarizar criterios de validación y fortalecer el control y seguimiento de las actividades, contribuyendo a una gestión más eficiente y confiable de los recursos.

La Actividad 6 se enfocó en el diseño y desarrollo de scripts para la automatización de tareas operativas relacionadas con el seguimiento de actividades, la validación de dependencias y la programación de recordatorios, utilizando una base de datos tabular implementada en una hoja de cálculo. Con el fin de cumplir con los acuerdos de confidencialidad establecidos,

todas las implementaciones se realizaron sobre registros ficticios, simulando escenarios reales sin exponer información sensible.

El presente reporte técnico documenta el proceso seguido para la identificación de las tareas a automatizar, el diseño general de las soluciones propuestas, la implementación de los scripts y la validación de sus resultados. Asimismo, el documento evidencia el desarrollo de competencias en análisis de procesos, lógica de programación, automatización y documentación técnica, alineadas con el perfil profesional de Ingeniería en Sistemas y con los objetivos formativos de las prácticas profesionales.

Descripción de la tarea a automatizar

Durante el desarrollo de las prácticas profesionales se identificaron diversas tareas operativas relacionadas con el seguimiento y control de actividades técnicas, las cuales se realizaban de forma manual a partir de una base de datos tabular implementada en una hoja de cálculo. Dichas tareas incluían la validación de dependencias entre actividades, la actualización del estatus de ejecución y la programación manual de recordatorios para el seguimiento de tareas.

El proceso manual original requería que el usuario revisara de manera individual cada registro para verificar si una tarea podía continuar su ejecución, considerando el estatus de una o más tareas previas. Esta validación implicaba consultar múltiples filas, comparar identificadores y determinar de forma manual si se cumplían los requisitos necesarios, lo cual incrementaba el tiempo invertido y la probabilidad de errores, especialmente cuando el volumen de tareas era elevado.

Adicionalmente, la programación de recordatorios y actividades de seguimiento se realizaba de forma manual, requiriendo que el usuario copiara información relevante de la hoja de cálculo y la registrara en herramientas externas de gestión del tiempo. Este procedimiento representaba una duplicidad de esfuerzos y una dependencia directa de la atención del usuario para evitar omisiones o inconsistencias en la planificación.

Con base en lo anterior, se propuso la automatización de dos tareas operativas principales:

- La verificación automática del estado de dependencias entre tareas, con base en el estatus de ejecución de tareas previas.
- El agendamiento automático de tareas seleccionadas, generando eventos y pendientes a partir de la información registrada en la hoja de cálculo.

Para la implementación de estas automatizaciones se desarrollaron scripts utilizando un entorno de scripting integrado a la hoja de cálculo, trabajando exclusivamente con datos

ficticios (dummy) y estructuras simuladas, con el objetivo de representar escenarios reales sin comprometer información sensible.

La automatización del control de dependencias se basa en la lectura de los registros almacenados en la hoja, la construcción de un mapa interno de estados y la evaluación de reglas lógicas para determinar si una tarea se encuentra habilitada o bloqueada. El siguiente fragmento de código ilustra la lógica general utilizada para construir la referencia de estados actuales:

```
let mapaEstados = {};  
for (let i = 1; i < data.length; i++) {  
  let id = data[i][COL_ID];  
  let estado = data[i][COL_ESTADO_ACTUAL];  
  if (id) {  
    mapaEstados[id] = estado;  
  }  
}
```

Posteriormente, el script recorre los registros para validar si una tarea depende de otra y asigna automáticamente el estado correspondiente, utilizando valores estandarizados como *OK*, *BLOQUEADO* o *N/A*, acompañados de indicadores visuales para facilitar su interpretación:

```
if (estadoPadre === "Completado") {  
  celdaEstadoDep.setValue("OK");  
} else {  
  celdaEstadoDep.setValue("BLOQUEADO");  
}
```

En cuanto al agendamiento automático de tareas, el proceso se activa a partir de la fila seleccionada por el usuario. El script valida que la ejecución se realice en el contexto correcto, extrae los campos mínimos requeridos —identificador, nombre de la tarea, responsable y fecha de inicio— y genera de manera automática un evento y una tarea de seguimiento. El siguiente fragmento representa la extracción de datos clave desde la hoja de cálculo:

```
const idTarea = datos[0];  
const nombreTarea = datos[2];  
const fechaInicio = datos[8];  
const responsable = datos[10];
```

Antes de ejecutar el agendamiento, el script incorpora validaciones básicas para garantizar la consistencia de la información, como la verificación de la existencia de una fecha de inicio. En caso de cumplirse las condiciones, se procede a la creación del evento y la tarea correspondiente; de lo contrario, se muestra un mensaje de advertencia al usuario.

En conjunto, estas tareas automatizadas sustituyen procesos manuales repetitivos por flujos controlados y reproducibles, permitiendo mejorar la eficiencia operativa, reducir errores y fortalecer el seguimiento de las actividades, manteniendo un enfoque responsable mediante el uso de datos simulados.

Metodología de desarrollo del script

La metodología empleada para el desarrollo de los scripts de automatización se basó en un enfoque sistemático y progresivo, orientado a garantizar la correcta comprensión del proceso operativo, la definición clara de reglas lógicas y la validación de resultados antes de su ejecución. Este enfoque permitió construir soluciones funcionales y controladas, manteniendo un equilibrio entre simplicidad, eficiencia y claridad del flujo de ejecución.

En una primera etapa se llevó a cabo la identificación y delimitación de las tareas operativas susceptibles de automatización. Para ello, se analizó el flujo de trabajo existente en la hoja de cálculo, identificando actividades repetitivas que dependían de validaciones manuales y de la interpretación del estatus de tareas previas. Este análisis permitió establecer dos procesos clave: el control de dependencias entre tareas y el agendamiento de actividades a partir de registros tabulares.

Posteriormente, se realizó el diseño lógico de las soluciones, definiendo las entradas, procesos y salidas de cada script. En el caso del control de dependencias, se estableció como entrada principal el conjunto de registros de la hoja de cálculo, mientras que el proceso consistió en la construcción de un mapa de estados y la evaluación de reglas condicionales para determinar si una tarea se encontraba habilitada o bloqueada. La salida del proceso se materializó en la actualización automática del estado de dependencia y su correspondiente indicador visual.

Para el agendamiento automático de tareas, el diseño lógico contempló la lectura de una fila seleccionada por el usuario como punto de entrada, la validación de campos obligatorios —como la fecha de inicio y el contexto de ejecución— y la generación de un evento y una tarea de seguimiento como resultado. Este diseño permitió asegurar que la automatización se ejecutara únicamente bajo condiciones válidas, evitando registros incompletos o inconsistentes.

Una vez definido el diseño lógico, se procedió a la implementación de los scripts utilizando un entorno de scripting integrado a la hoja de cálculo. Durante esta etapa se adoptaron buenas

prácticas de desarrollo, tales como la separación de funciones, el uso de constantes para el mapeo de columnas y la inclusión de comentarios descriptivos que facilitan la comprensión y el mantenimiento del código. El siguiente fragmento ilustra el uso de constantes para el mapeo de columnas, lo cual permite adaptar el script ante cambios en la estructura de la hoja:

```
const COL_ID = 0;  
const COL_TIENE_DEP = 4;  
const COL_ID_DEP = 5;  
const COL_ESTADO_DEP = 6;  
const COL_ESTADO_ACTUAL = 7;
```

Asimismo, se incorporaron mecanismos de validación y manejo de errores para controlar escenarios comunes de fallo, como la ejecución del script en una hoja incorrecta, la selección del encabezado en lugar de una fila de datos o la ausencia de campos obligatorios. Estos controles se implementaron mediante estructuras condicionales y bloques de manejo de excepciones, garantizando una ejecución segura y predecible.

Finalmente, la metodología incluyó una fase de pruebas utilizando exclusivamente datos ficticios (dummy), con el objetivo de verificar el comportamiento de los scripts ante distintos escenarios, tales como tareas con dependencias satisfechas, tareas bloqueadas y registros incompletos. Los resultados de estas pruebas permitieron ajustar la lógica implementada y confirmar que las automatizaciones cumplieran con los objetivos planteados, sin comprometer información sensible ni interferir con procesos reales.

Diseño general de la solución

El diseño general de la solución se estructuró bajo un enfoque modular, con el objetivo de separar claramente las responsabilidades de cada automatización y facilitar su comprensión, mantenimiento y posible extensión. La solución se apoya en una base de datos tabular implementada en una hoja de cálculo, la cual funciona como repositorio central de información operativa y punto de interacción con el usuario.

A nivel lógico, la solución se compone de dos módulos principales de automatización: el módulo de control de dependencias y el módulo de agendamiento automático de tareas. Ambos módulos operan sobre la misma fuente de datos, pero ejecutan procesos independientes y complementarios, activados bajo diferentes contextos de uso.

El módulo de control de dependencias tiene como propósito evaluar de forma automática si una tarea puede continuar su ejecución con base en el estatus de una tarea previa. Para

ello, el script realiza una lectura completa de los registros, construye una estructura interna que relaciona identificadores de tareas con su estado actual y, posteriormente, recorre nuevamente los registros para aplicar reglas condicionales. El resultado de esta evaluación se refleja directamente en la hoja de cálculo mediante la actualización del campo correspondiente al estado de dependencia y el uso de indicadores visuales que facilitan su interpretación.

Desde el punto de vista funcional, este módulo transforma un proceso manual de validación cruzada —que requería la revisión individual de múltiples registros— en una operación sistemática y reproducible, reduciendo la carga operativa y minimizando errores derivados de interpretaciones subjetivas.

Por su parte, el módulo de agendamiento automático de tareas está diseñado para operar de manera puntual sobre un registro seleccionado por el usuario. El flujo inicia cuando el usuario selecciona una fila específica y ejecuta la acción correspondiente desde la interfaz de la hoja de cálculo. El script valida que la ejecución se realice en el contexto adecuado, extrae los datos mínimos necesarios y genera automáticamente un evento y una tarea de seguimiento, utilizando la información ya registrada en la base tabular.

El diseño de este módulo prioriza la validación previa de condiciones mínimas, tales como la existencia de una fecha de inicio y la correcta selección de una fila de datos, con el fin de evitar la creación de registros incompletos. Una vez superadas estas validaciones, el proceso de agendamiento se ejecuta de forma transparente para el usuario, quien recibe una confirmación visual al finalizar la operación.

Ambos módulos incorporan mecanismos de control y retroalimentación al usuario, tales como mensajes informativos y alertas ante condiciones no válidas. Asimismo, el diseño contempla la integración de acciones personalizadas en la interfaz de la hoja de cálculo, mediante menús y botones, lo que permite ejecutar las automatizaciones sin necesidad de acceder directamente al entorno de scripting.

En conjunto, el diseño general de la solución establece un flujo claro de interacción usuario-sistema, en el cual la hoja de cálculo actúa como interfaz principal, los scripts como motor de automatización y los servicios auxiliares como soporte para la generación de recordatorios y seguimiento. Esta arquitectura lógica permite simular un entorno operativo real, manteniendo un nivel de abstracción adecuado y utilizando exclusivamente datos ficticios para cumplir con los acuerdos de confidencialidad establecidos.

Resultados obtenidos

Los resultados obtenidos a partir de la implementación de los scripts de automatización evidencian una mejora significativa en el control, seguimiento y ejecución de tareas opera-

tivas previamente realizadas de forma manual. Las automatizaciones desarrolladas fueron probadas utilizando exclusivamente registros ficticios (dummy), lo que permitió validar su funcionamiento en escenarios simulados sin comprometer información sensible.

En primer lugar, se logró la automatización del control de dependencias entre tareas, permitiendo identificar de manera inmediata si una actividad se encontraba habilitada o bloqueada en función del estatus de una tarea previa. Como se observa en las evidencias gráficas del apéndice, el sistema actualiza automáticamente el estado de dependencia de cada registro, asignando valores estandarizados y acompañándolos de indicadores visuales que facilitan su interpretación. Esta funcionalidad elimina la necesidad de realizar verificaciones manuales cruzadas entre múltiples filas y reduce el riesgo de errores por omisión o interpretación incorrecta.

Asimismo, se documentó la correcta ejecución de las reglas de dependencia definidas, las cuales se encuentran respaldadas por una hoja auxiliar de restricciones operativas. Dichas reglas permiten simular escenarios reales de bloqueo y habilitación, proporcionando trazabilidad y consistencia en la aplicación de criterios para la validación del flujo de trabajo.

En cuanto al agendamiento automático de tareas, los resultados muestran que el script es capaz de convertir de forma efectiva un registro seleccionado en un evento y una tarea de seguimiento, utilizando la información previamente capturada en la hoja de cálculo. Las evidencias muestran la correcta validación del contexto de ejecución, la extracción de datos clave y la generación automática de los elementos correspondientes, así como la notificación visual al usuario al finalizar el proceso.

Adicionalmente, se comprobó el correcto funcionamiento de los mecanismos de validación y manejo de errores implementados. En los casos en que no se cumplen las condiciones mínimas para la ejecución —como la ausencia de una fecha de inicio o la selección de una fila no válida— el sistema detiene el proceso y muestra mensajes informativos, evitando la creación de registros incompletos o inconsistentes.

Finalmente, las evidencias gráficas correspondientes a la herramienta de calendario confirman la creación exitosa de los eventos y tareas generados por el script, validando la integración del proceso automatizado con los mecanismos de seguimiento del usuario. En conjunto, los resultados demuestran que las automatizaciones cumplen con los objetivos planteados, operan de manera controlada y reproducible, y aportan mejoras tangibles en la eficiencia y confiabilidad del proceso operativo simulado.

Beneficios de la automatización

La automatización de las tareas operativas implementadas aportó beneficios claros en términos de eficiencia, control y confiabilidad del proceso simulado. Al sustituir actividades manuales repetitivas por scripts estructurados, se logró optimizar el flujo de trabajo y reducir la dependencia de la intervención constante del usuario.

Uno de los principales beneficios fue la reducción del tiempo dedicado a la validación de dependencias entre tareas. La verificación automática del estatus permitió identificar de manera inmediata si una actividad se encontraba habilitada o bloqueada, eliminando la necesidad de revisar manualmente múltiples registros y disminuyendo la carga operativa asociada a esta tarea. Asimismo, el uso de indicadores visuales contribuyó a una interpretación más rápida y clara del estado del proceso.

Otro beneficio relevante fue la disminución de errores derivados de la ejecución manual. La estandarización de reglas lógicas y la incorporación de validaciones previas aseguraron que las automatizaciones solo se ejecutaran bajo condiciones válidas, evitando inconsistencias como la creación de recordatorios sin fecha definida o la programación de tareas en contextos incorrectos. Este enfoque preventivo fortaleció la confiabilidad de la información generada.

La automatización del agendamiento de tareas también favoreció una mejor organización y seguimiento de las actividades. Al generar de forma automática eventos y tareas a partir de los registros existentes, se eliminó la duplicidad de esfuerzos y se aseguró la sincronización entre la base de datos tabular y las herramientas de seguimiento del usuario. Esto permitió una planificación más consistente y redujo el riesgo de omisiones en el control de pendientes.

Desde una perspectiva operativa, la solución desarrollada facilitó la trazabilidad de las actividades, ya que los estados, dependencias y acciones ejecutadas quedan reflejados directamente en la hoja de cálculo utilizada como repositorio central. Esta característica resulta especialmente valiosa para el seguimiento del avance y la identificación oportuna de bloqueos dentro del flujo de trabajo.

Finalmente, desde el punto de vista formativo y profesional, la actividad permitió fortalecer competencias relacionadas con el análisis de procesos, la lógica de programación, el diseño de soluciones automatizadas y la documentación técnica. La experiencia adquirida demuestra la aplicabilidad de la automatización como una herramienta efectiva para mejorar procesos operativos, manteniendo siempre un enfoque responsable mediante el uso de datos ficticios y el cumplimiento de los acuerdos de confidencialidad.

Conclusiones

El desarrollo de la Actividad 6 permitió demostrar la aplicación práctica de la automatización como una estrategia efectiva para mejorar procesos operativos caracterizados por tareas repetitivas y validaciones manuales. A través del diseño e implementación de scripts orientados al control de dependencias y al agendamiento automático de tareas, fue posible transformar flujos de trabajo manuales en procesos estructurados, reproducibles y controlados.

La automatización del seguimiento de dependencias evidenció una mejora significativa en la eficiencia operativa, al reducir el tiempo necesario para identificar bloqueos y habilitaciones entre tareas. Asimismo, la incorporación de reglas estandarizadas, validaciones previas e indicadores visuales fortaleció la confiabilidad de la información y facilitó la interpretación del estado del proceso, disminuyendo la probabilidad de errores derivados de la intervención manual.

Por su parte, el agendamiento automático de tareas permitió optimizar la planificación y el seguimiento de actividades, eliminando la duplicidad de esfuerzos asociada al registro manual de recordatorios. La correcta integración entre la base de datos tabular y las herramientas de gestión del tiempo contribuyó a una mayor trazabilidad de las actividades y a una mejor organización de los pendientes operativos.

Desde el punto de vista técnico, la actividad fortaleció competencias relacionadas con el análisis de procesos, la lógica de programación, el diseño modular de soluciones automatizadas y la implementación de mecanismos de validación y manejo de errores. Asimismo, se reforzaron habilidades de documentación técnica, al describir de forma clara y estructurada el funcionamiento de las soluciones desarrolladas.

Finalmente, el uso de datos ficticios permitió simular escenarios reales de operación sin comprometer información sensible, cumpliendo con los acuerdos de confidencialidad establecidos. En conjunto, los resultados obtenidos confirman la relevancia de la automatización como una herramienta clave para la mejora continua de procesos operativos y para el fortalecimiento del perfil profesional en el área de Ingeniería en Sistemas.

Referencias

- DataCamp. (n.d.). *Python try-except: Exception handling explained*. Recuperado de: <https://www.datacamp.com/tutorial/python-try-except>
- DataCamp. (n.d.). *Python f-strings: Everything you need to know*. Recuperado de: <https://www.datacamp.com/tutorial/python-f-string>

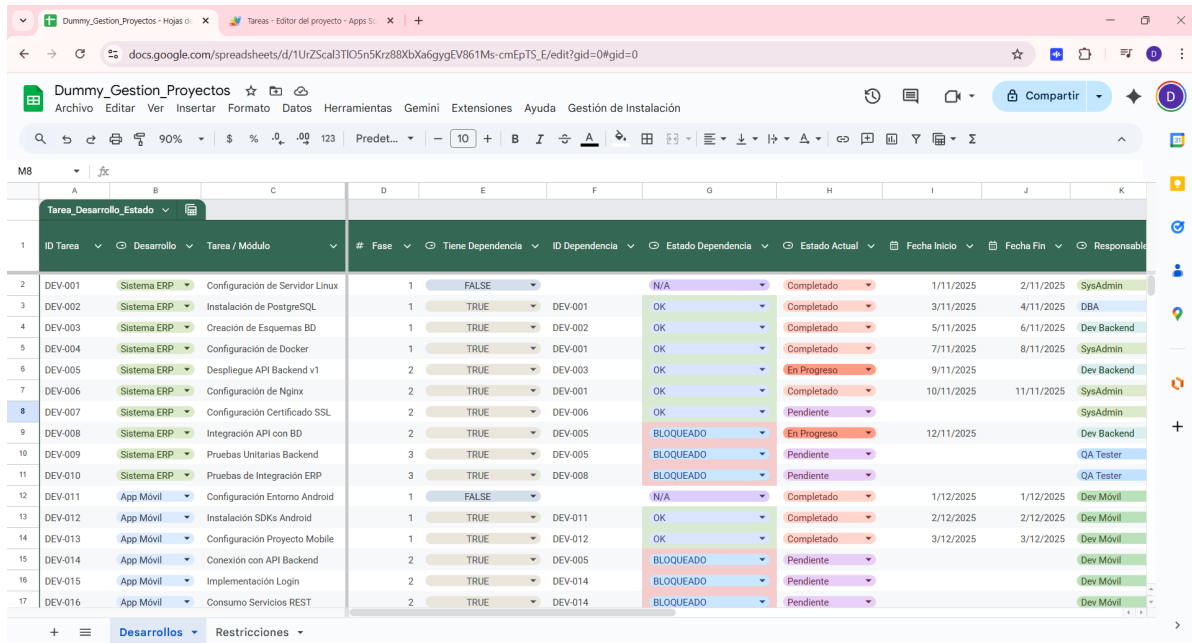
- Google. (n.d.). *Importar y exportar archivos*. Recuperado de: <https://support.google.com/docs/answer/3093340?hl=es-419>
- Google. (n.d.). *Convertir archivos a Google Docs*. Recuperado de: <https://support.google.com/docs/answer/3093197?hl=es-419>
- Google Cloud. (n.d.). *Cómo crear campos calculados en Looker Studio*. Recuperado de: <https://docs.cloud.google.com/looker/docs/studio/tutorial-create-calculated-fields-hl=es-419>
- Google Cloud. (n.d.). *Variables de resultado de consulta en Looker Studio*. Recuperado de: <https://docs.cloud.google.com/looker/docs/studio/query-result-variables?hl=es-419>
- Google Cloud. (n.d.). *Agregar, editar y solucionar problemas con campos calculados en Looker Studio*. Recuperado de: <https://docs.cloud.google.com/looker/docs/studio/add-edit-and-troubleshoot-calculated-fields?hl=es-419>
- Google Workspace Developers. (n.d.). *Google Apps Script overview*. Recuperado de: <https://developers.google.com/apps-script/overview>
- Google Workspace Developers. (n.d.). *Best practices for Google Apps Script*. Recuperado de: <https://developers.google.com/apps-script/guides/support/best-practices>
- Google Workspace Developers. (n.d.). *Custom menus in Google Sheets*. Recuperado de: <https://developers.google.com/apps-script/guides/menus>
- Google Workspace Developers. (n.d.). *Spreadsheet service*. Recuperado de: <https://developers.google.com/apps-script/reference/spreadsheet>
- Google Workspace Developers. (n.d.). *Calendar service*. Recuperado de: <https://developers.google.com/apps-script/reference/calendar>
- Google Workspace Developers. (n.d.). *Tasks API overview*. Recuperado de: <https://developers.google.com/tasks>
- Python Software Foundation. (n.d.). *Built-in exceptions*. Recuperado de: <https://docs.python.org/3/library/exceptions.html>
- Python Software Foundation. (n.d.). *PEP 8 – Style Guide for Python Code*. Recuperado de: <https://peps.python.org/pep-0008/>

- Real Python. (n.d.). *Python Exceptions: An Introduction*. Recuperado de: <https://realpython.com/python-exceptions/>

A. Evidencias gráficas de la Actividad 6

El presente apéndice integra las evidencias gráficas correspondientes a la Actividad 6: *Desarrollo de scripts para automatización de tareas operativas*. Las capturas presentadas documentan de forma secuencial el proceso de implementación, ejecución y validación de las automatizaciones desarrolladas sobre una base de datos tabular con registros ficticios.

Las evidencias se presentan en el mismo orden en que fueron generadas durante el desarrollo de la actividad, con el propósito de reflejar de manera clara el flujo de trabajo seguido. Todas las implementaciones se realizaron utilizando datos dummy, a fin de simular escenarios reales sin comprometer información sensible ni violar acuerdos de confidencialidad.



ID Tarea	Desarrollo	Tarea / Módulo	# Fase	Tiene Dependencia	ID Dependencia	Estado Dependencia	Estado Actual	Fecha Inicio	Fecha Fin	Responsable
DEV-001	Sistema ERP	Configuración de Servidor Linux	1	FALSE		N/A	Completado	1/11/2025	2/11/2025	SysAdmin
DEV-002	Sistema ERP	Instalación de PostgreSQL	1	TRUE	DEV-001	OK	Completado	3/11/2025	4/11/2025	DBA
DEV-003	Sistema ERP	Creación de Esquemas BD	1	TRUE	DEV-002	OK	Completado	5/11/2025	6/11/2025	Dev Backend
DEV-004	Sistema ERP	Configuración de Docker	1	TRUE	DEV-001	OK	Completado	7/11/2025	8/11/2025	SysAdmin
DEV-005	Sistema ERP	Despliegue API Backend v1	2	TRUE	DEV-003	OK	En Progreso	9/11/2025		Dev Backend
DEV-006	Sistema ERP	Configuración de Nginx	2	TRUE	DEV-001	OK	Completado	10/11/2025	11/11/2025	SysAdmin
DEV-007	Sistema ERP	Configuración Certificado SSL	2	TRUE	DEV-006	OK	Pendiente			SysAdmin
DEV-008	Sistema ERP	Integración API con BD	2	TRUE	DEV-005	BLOQUEADO	En Progreso	12/11/2025		Dev Backend
DEV-009	Sistema ERP	Pruebas Unitarias Backend	3	TRUE	DEV-005	BLOQUEADO	Pendiente			QA Tester
DEV-010	Sistema ERP	Pruebas de Integración ERP	3	TRUE	DEV-008	BLOQUEADO	Pendiente			QA Tester
DEV-011	App Móvil	Configuración Entorno Android	1	FALSE		N/A	Completado	1/12/2025	1/12/2025	Dev Móvil
DEV-012	App Móvil	Instalación SDKs Android	1	TRUE	DEV-011	OK	Completado	2/12/2025	2/12/2025	Dev Móvil
DEV-013	App Móvil	Configuración Proyecto Mobile	1	TRUE	DEV-012	OK	Completado	3/12/2025	3/12/2025	Dev Móvil
DEV-014	App Móvil	Conexión con API Backend	2	TRUE	DEV-005	BLOQUEADO	Pendiente			Dev Móvil
DEV-015	App Móvil	Implementación Login	2	TRUE	DEV-014	BLOQUEADO	Pendiente			Dev Móvil
DEV-016	App Móvil	Consumo Servicios REST	2	TRUE	DEV-014	BLOQUEADO	Pendiente			Dev Móvil

Figura 1: Vista general de la hoja de control de tareas operativas con registros ficticios, incluyendo campos de dependencia y estatus, utilizada como base para la automatización del seguimiento.

ID Regla	Tecnología Base	Tecnología Dependiente	Tipo de Bloqueo	Mensaje de Error
R-01	Servidor Linux	Docker	Crítico	No se puede instalar Docker sin sistema operativo base configurado
R-02	PostgreSQL	Esquemas BD	Crítico	La base de datos no está disponible o no responde
R-03	API Backend	App Móvil	Funcional	La aplicación no puede consumir servicios sin API activa
R-04	Nginx	Certificado SSL	Seguridad	No es posible habilitar HTTPS sin certificado válido
R-05	Node.js	Librerías NPM	Crítico	El entorno de ejecución requerido no está instalado
R-06	API Backend	Pruebas de Integración	Funcional	Las pruebas fallan si los servicios no están desplegados
R-07	Base de Datos	Pruebas Funcionales	Funcional	No existen datos disponibles para pruebas
R-08	Docker	Servicios Backend	Crítico	Los contenedores no están en ejecución
R-09	Firewall	Servicios Externos	Seguridad	Acceso bloqueado por reglas de red
R-10	Backend	Login App	Funcional	El servicio de autenticación no responde
R-11	API Gateway	App Móvil	Funcional	No se puede enrutar tráfico a la aplicación
R-12	Certificados	Servicios Web	Seguridad	Certificado expirado o inválido
R-13	Base de Datos	Optimización SQL	Funcional	No se puede optimizar sin acceso a métricas
R-14	Servicios REST	Pruebas Rendimiento	Funcional	Servicios no disponibles para pruebas de carga
R-15	Pipeline CI/CD	Despliegue Producción	Crítico	El pipeline presenta errores de compilación

Figura 2: Catálogo de reglas y restricciones operativas documentado en una hoja auxiliar, utilizado para estandarizar criterios de bloqueo y validación de dependencias.

```

1  /**
2   * Función para actualizar el estado de dependencias automáticamente.
3   * Verifica si la tarea "padre" está completada.
4   */
5  function actualizarEstadoInstalacion() {
6    const sheet = SpreadsheetApp.getActiveSpreadsheet().getSheetByName("Desarrollos");
7    if (!sheet) {
8      SpreadsheetApp.getUi().alert("No se encontró la hoja 'Desarrollos'");
9      return;
10   }
11
12   const dataRange = sheet.getDataRange();
13   const data = dataRange.getValues();
14
15   // Mapeo de columnas (A=0, B=1, etc.)
16   const COL_ID = 0; // A: ID Tarea
17   const COL_TIENE_DEP = 4; // E: Tiene Dependencia
18   const COL_ID_DEP = 5; // F: ID Dependencia
19   const COL_ESTADO_DEP = 6; // G: Estado Dependencia
20   const COL_ESTADO_ACTUAL = 7; // H: Estado Actual
21
22   // 1. Crear mapa de estados actuales
23   let mapaEstados = {};
24   for (let i = 1; i < data.length; i++) {
25     let id = data[i][COL_ID];
26     let estado = data[i][COL_ESTADO_ACTUAL];
27     if (id) mapaEstados[id] = estado;
28   }
29
30   // 2. Verificar bloqueos
31   for (let i = 1; i < data.length; i++) {
32     let tieneDep = data[i][COL_TIENE_DEP];
33     let idPadre = data[i][COL_ID_DEP];
34     let celdaEstadoDep = sheet.getRange(i + 1, COL_ESTADO_DEP + 1);
35

```

Figura 3: Implementación del script de control de dependencias, mostrando la lectura de registros y la construcción de una referencia interna de estatus por identificador.

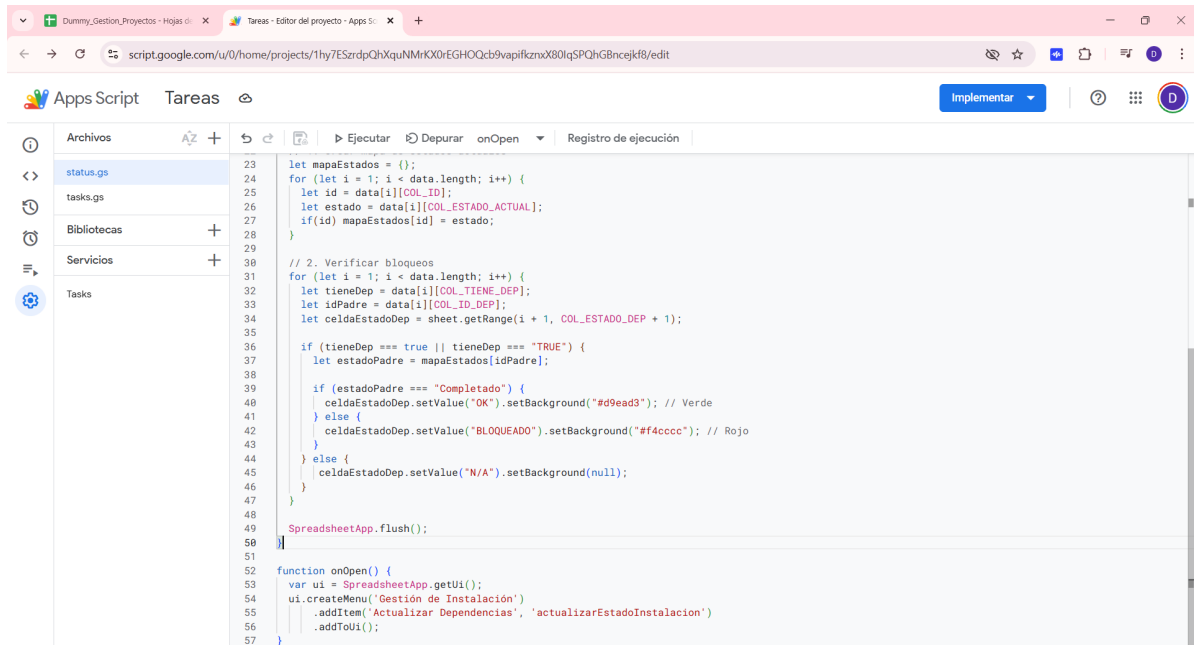


Figura 4: Lógica de validación de dependencias y asignación automática de estados, junto con la integración de una opción de menú para la ejecución controlada del script.

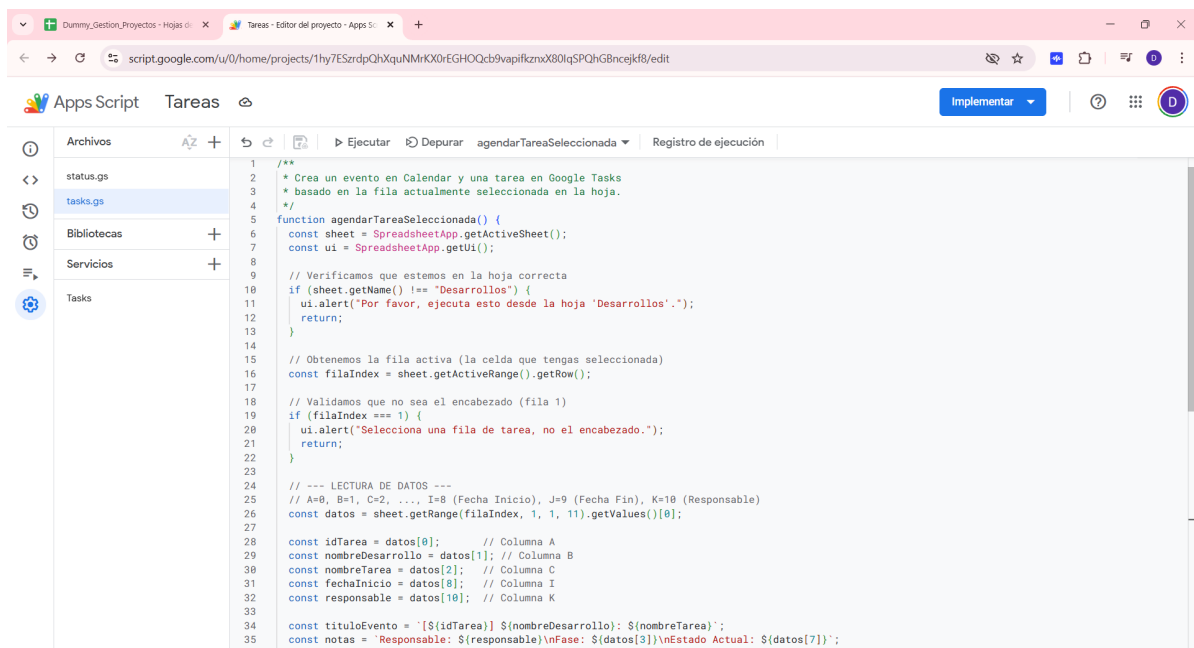


Figura 5: Implementación del script de agendamiento automático, mostrando la validación de contexto y la extracción de datos clave desde la fila seleccionada.

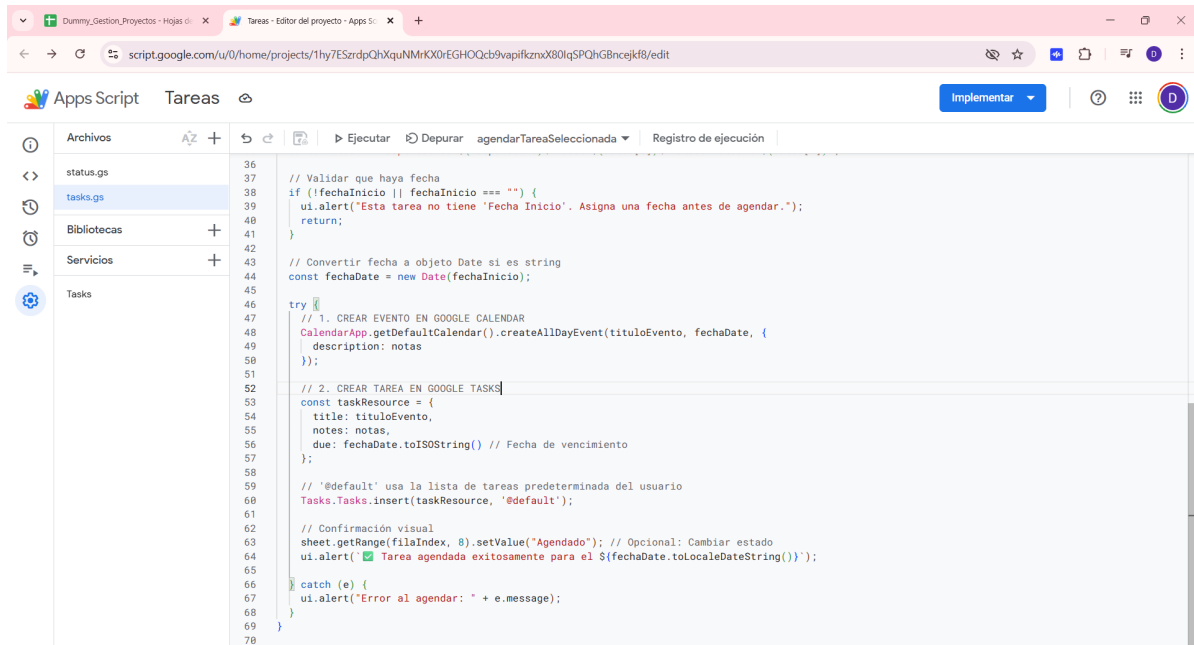


Figura 6: Ejecución del proceso de agendamiento automático, incluyendo la creación de eventos y tareas de seguimiento con manejo de excepciones y retroalimentación al usuario.

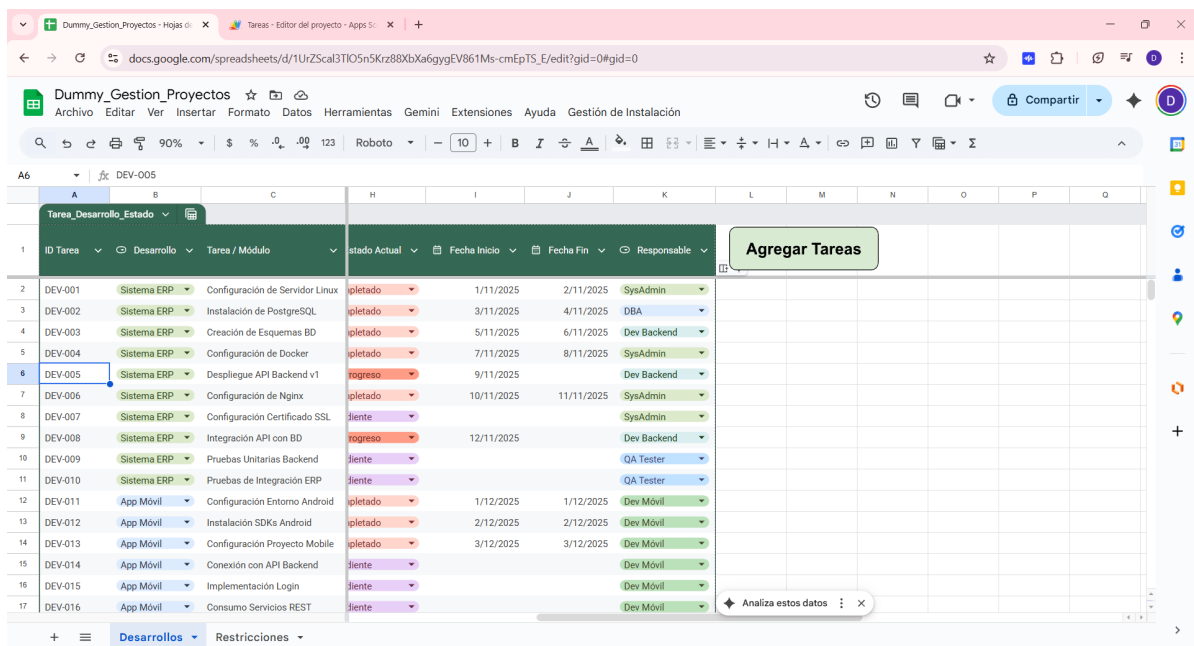


Figura 7: Integración de un control de ejecución en la hoja de cálculo mediante un botón de acción para disparar la automatización sobre la tarea seleccionada.

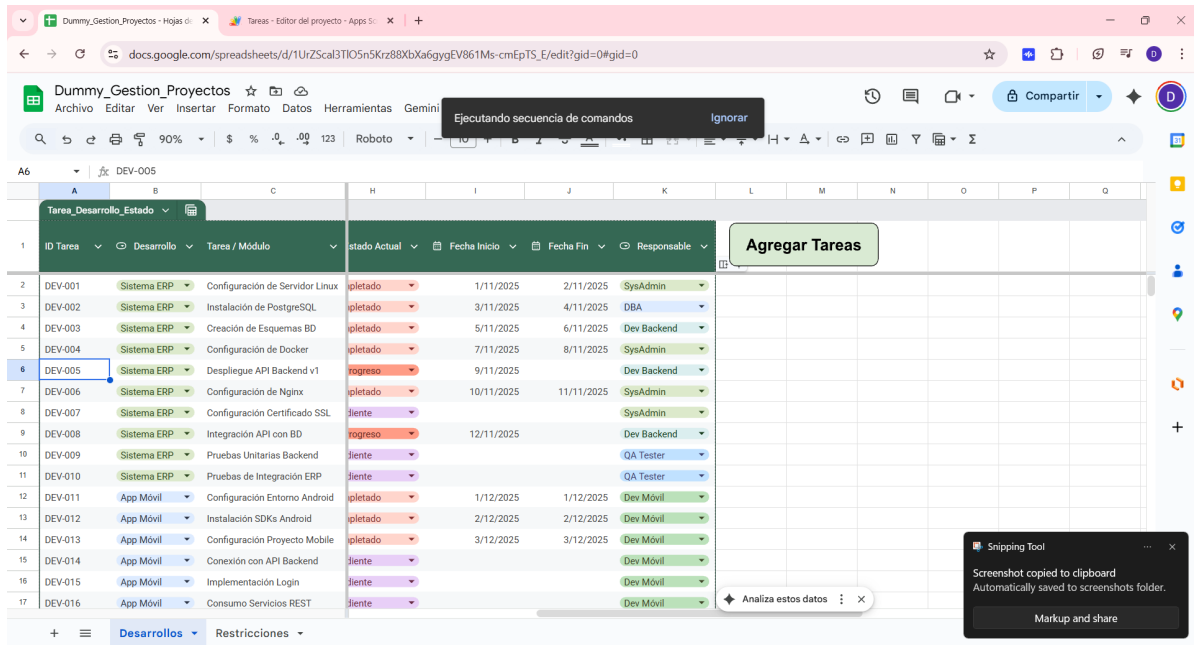


Figura 8: Evidencia de la ejecución del script desde la interfaz de usuario, mostrando el flujo de comandos activo durante la operación.

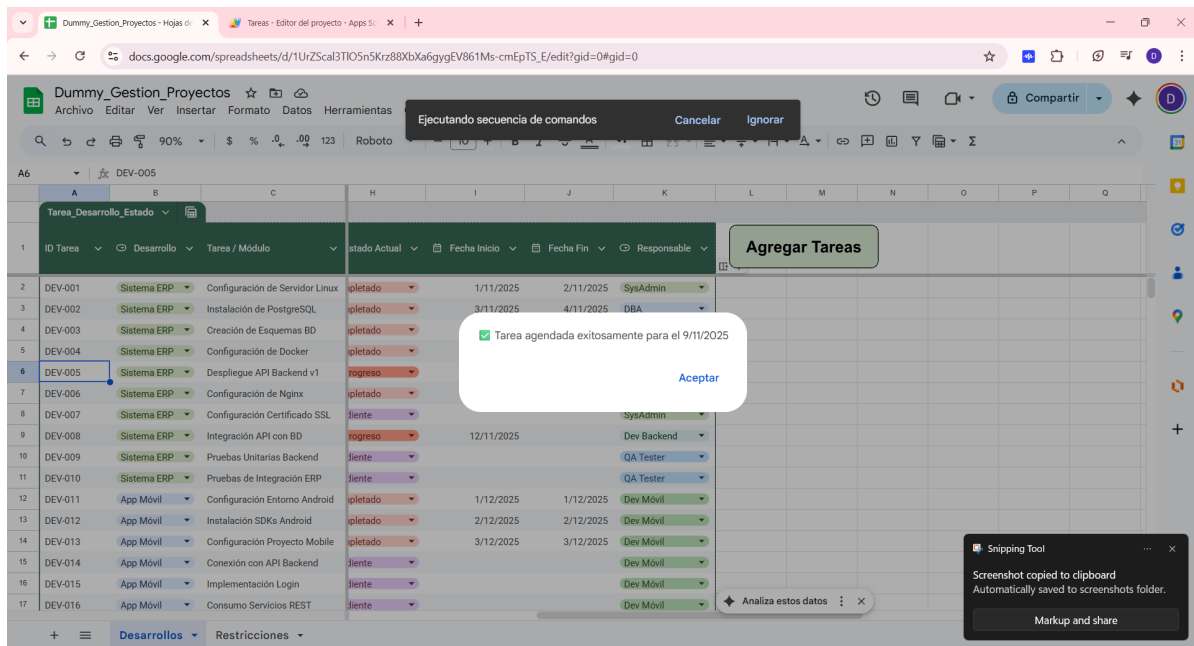


Figura 9: Confirmación visual de la ejecución exitosa del proceso automatizado, indicando el correcto agendamiento de la tarea.

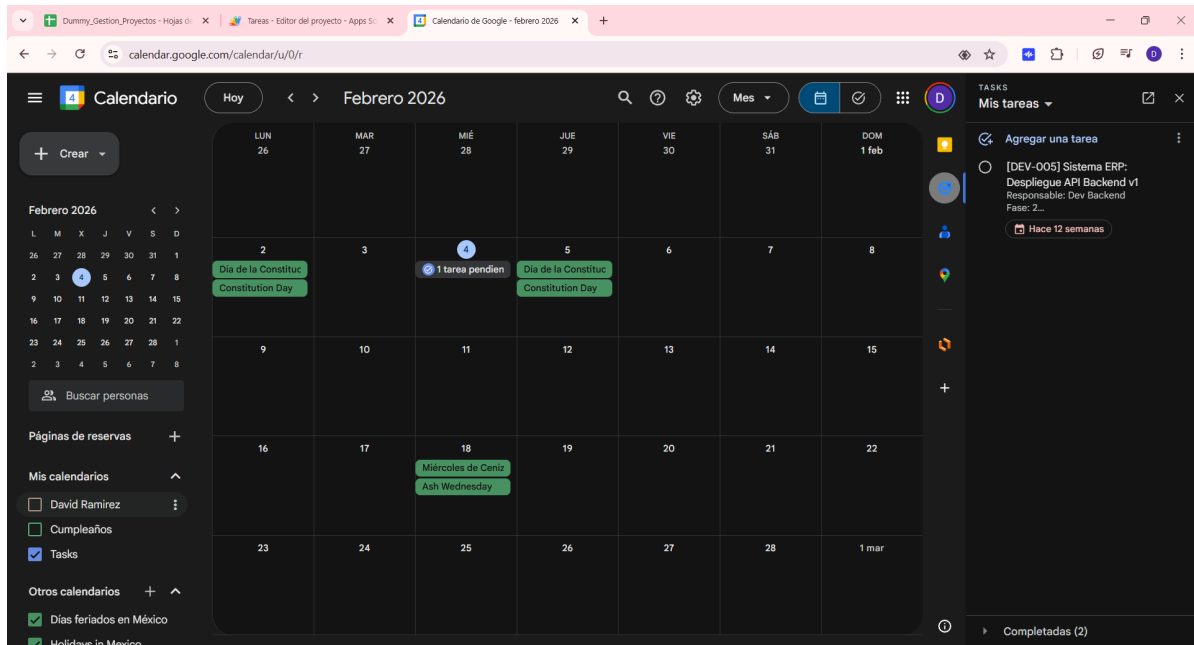


Figura 10: Validación del resultado en la herramienta de calendario, mostrando el evento y la tarea generados automáticamente a partir del registro seleccionado.